

Contents

Simulateur ADAS — Récapitulatif du projet	1
1. En une phrase	1
2. À quoi ça sert ?	1
3. Comment ça marche — Vue d'ensemble	2
4. Workflow du système	2
5. Les étapes déjà réalisées	3
6. Les outils utilisés	4
7. Ce qui reste à faire	5
8. En résumé	5
9. Contact	6

Simulateur ADAS — Récapitulatif du projet

Auteure : Molka Ammari **Date :** 2026-10-09 **Contexte :** Projet personnel — Simulateur SIL d'un système ADAS (Forward Collision Warning)

1. En une phrase

Un **simulateur logiciel** qui reproduit le fonctionnement d'un système d'assistance à la conduite (ADAS) : une caméra détecte un véhicule devant, un calculateur estime le risque de collision, et un tableau de bord affiche l'alerte. **Sans aucun matériel physique.**

2. À quoi ça sert ?

2.1 Le problème dans la vraie vie

Sur les voitures modernes, des systèmes comme le **freinage d'urgence automatique** existent déjà. Ils reposent sur :

- Une ou plusieurs **caméras**
- Un **radar** ou un **LiDAR**
- Un **calculateur embarqué** qui prend la décision
- Un **actionneur** (frein, alerte sonore)

Ces composants communiquent entre eux via un **bus CAN** : un réseau de communication industriel standard dans l'automobile.

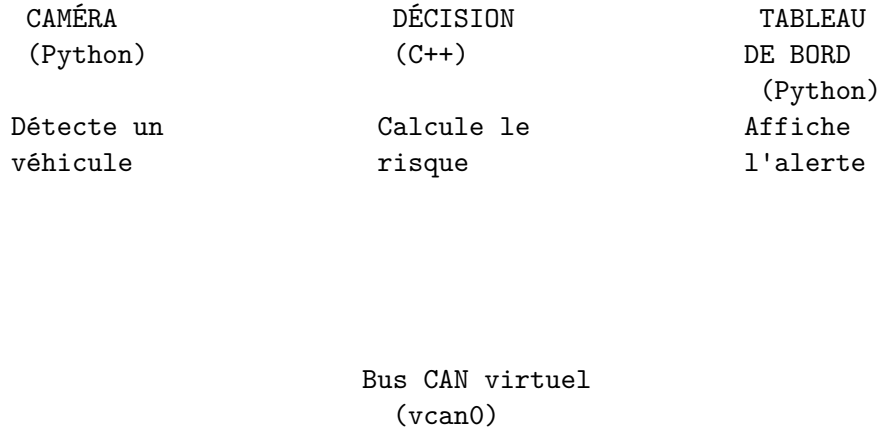
2.2 Notre simulation

Plutôt que d'acheter du matériel coûteux (caméras, radar, calculateur), **on simule tout sur un ordinateur** :

- La **caméra** → une image KITTI (dataset public de conduite automobile)
- Le **bus CAN** → une interface virtuelle **vcan0** fournie par Linux
- Le **calculateur** → un programme en C++
- Le **tableau de bord** → un programme Python

C'est ce qu'on appelle du **SIL (Software-in-the-Loop)** : on teste le logiciel sans le matériel. **C'est exactement ce que font les ingénieurs Bosch, Continental, Valeo** avant d'embarquer un système sur une vraie voiture.

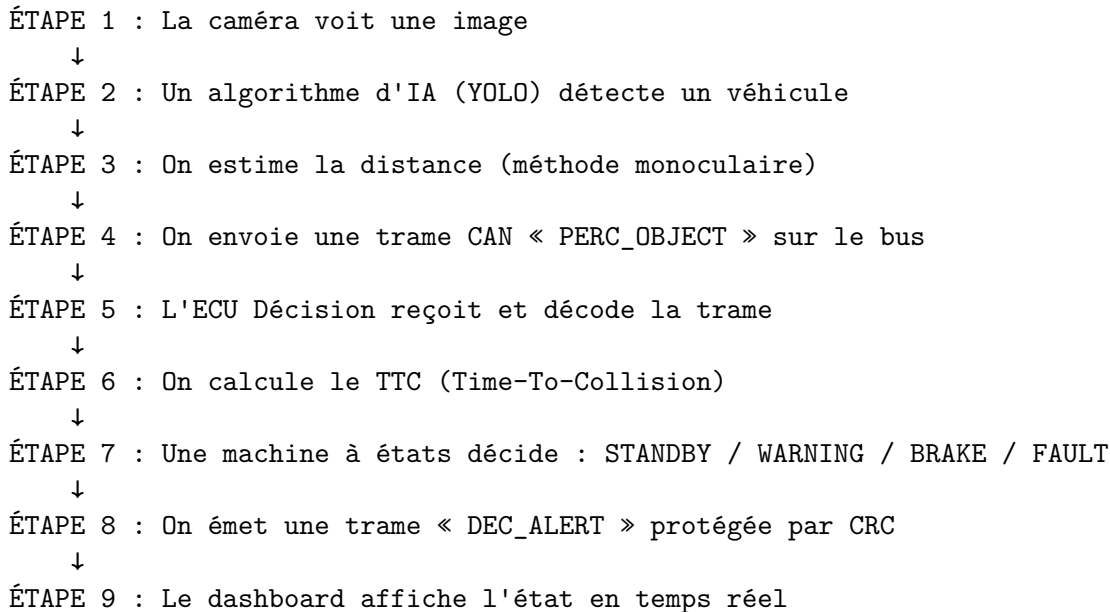
3. Comment ça marche — Vue d'ensemble



Analogie : imaginez un **jeu de dominos**. La caméra pose le premier domino (« j'ai vu une voiture »), le calculateur le fait tomber (« attention, collision dans 2 s »), et le tableau de bord réagit (« ALERTE ! »).

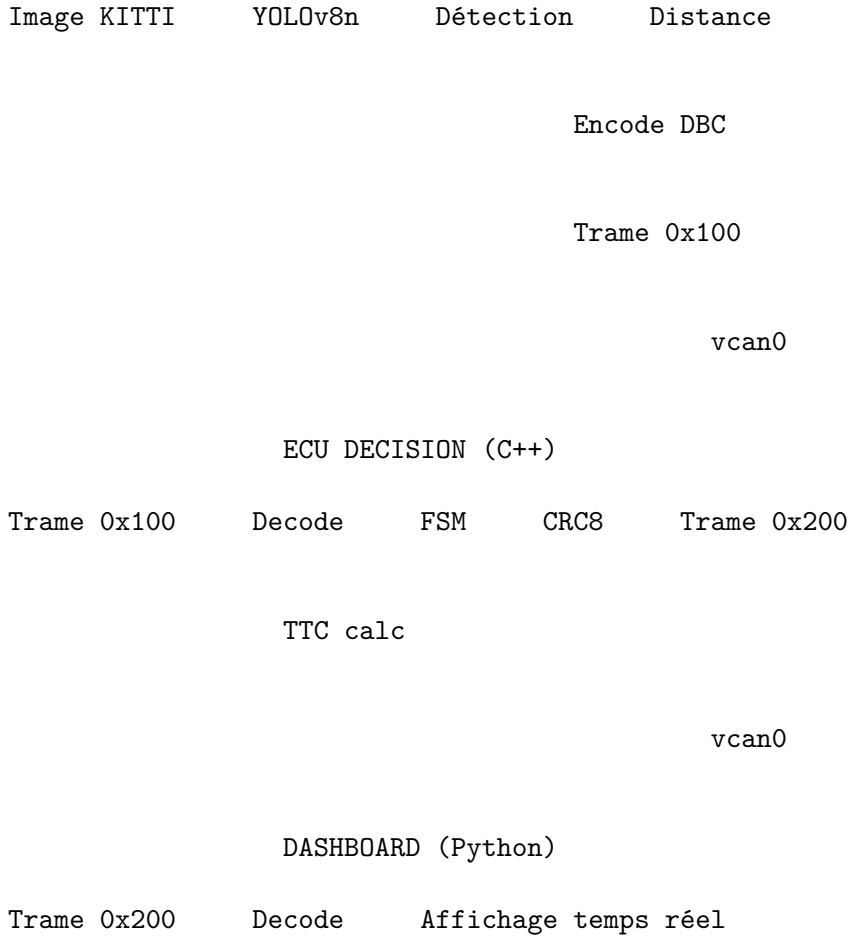
4. Workflow du système

4.1 Vue simplifiée (pour comprendre)



4.2 Workflow détaillé (technique)

ECU PERCEPTION (Python)



5. Les étapes déjà réalisées

Phase 0.2 — Bus CAN virtuel

Objectif : faire fonctionner un bus CAN virtuel (`vcan0`) sur Linux.

Résultat : - Module noyau `vcan` chargé - Interface `vcan0` créée et testée (loopback) - Communication CAN validée côté shell et Python

Phase 1 — Spécification

Objectif : définir **quoi** construire avant de coder.

Résultat : - **REQ-xxx** : 50+ exigences (perception, CAN, décision, sécurité, tests) - **DBC** : dictionnaire des 3 messages CAN (PERC_OBJECT, DEC_ALERT, DEC_HB) - **Architecture** : schéma des 3 ECU, choix techniques, limites assumées

Phase 2 — ECU Perception (Python)

Objectif : détecter un véhicule et estimer sa distance.

Résultat : - **YOLOv8n** : IA de détection d'objets (CPU) - **Distance monoculaire** : estimation à

partir de la taille du véhicule - **Émission CAN** : 4 modules Python (`detector`, `distance`, `can_tx`, `main`)
- **Test réussi** : une image KITTI → 1 trame CAN sur `vcan0`

Phase 3 — Tests DBC (Python)

Objectif : valider que le dictionnaire DBC est correct.

Résultat : - **30 tests** avec `pytest` - Vérification de l'encodage/décodage de chaque message - Garantie que les octets émis correspondent aux spécifications

Phase 4 — ECU Décision (C++)

Objectif : recevoir les détections, décider, émettre l'alerte.

Résultat : - **8 modules C++** (~2500 lignes) - **69 tests unitaires** (GoogleTest) - **Intégration bout-en-bout** validée : Python → C++ → émission CAN - **Machine à états** : OFF / STANDBY / WARNING / BRAKE / FAULT - **CRC8 E2E** : protection contre les corruptions

6. Les outils utilisés

6.1 Système et langages

Outil	Rôle
Ubuntu 22.04.5 LTS	Système d'exploitation (VM VirtualBox)
Python 3.10	Langage pour Perception + Dashboard
C++17	Langage pour l'ECU Décision
CMake 3.22	Build system C++

6.2 Bus CAN

Outil	Rôle
SocketCAN	API Linux native pour CAN
vcan	Module noyau pour bus CAN virtuel
can-utils	Outils ligne de commande (<code>cansend</code> , <code>candump</code>)

6.3 Intelligence artificielle

Outil	Rôle
YOLOv8n	Détection d'objets (ultralitics)
PyTorch (CPU)	Framework IA sous-jacent
OpenCV	Traitement d'image

6.4 Tests

Outil	Rôle
pytest	Tests Python

Outil	Rôle
GoogleTest	Tests C++

6.5 Communication et protocole

Outil	Rôle
cantools	Encodage/décodage DBC (Python)
python-can	Interface CAN Python
DBC	Format standard des messages CAN

7. Ce qui reste à faire

Phase 5 — Dashboard (Python)

Objectif : afficher l'état du système en temps réel.

Ce qui sera fait : - Lecture des trames DEC_ALERT (0x200) - Affichage clair : état courant, TTC, compteur
- Peut-être une interface graphique (TUI ou web)

Durée estimée : 1–2 h

Phase 6 — Validation SIL + injection de défauts

Objectif : tester la robustesse du système.

Ce qui sera fait : - Injection de **défauts** : perte de trame, CRC corrompu, compteur figé - Vérification que la FSM passe bien en **FAULT** - Scénarios complets avec plusieurs images - **Rapport de validation**

Durée estimée : 2–3 h

Phase 7 — Valorisation

Objectif : rendre le projet présentable.

Ce qui sera fait : - **README final** : présentation du projet, installation, démo - **Pitch** : présentation orale (2–3 min) - **Post LinkedIn** : mise en avant du projet - **Vidéo démo** (optionnel)

Durée estimée : 2–3 h

8. En résumé

Élément	Valeur
Avancement global	~75 %
Phases terminées	5 / 7
Lignes de code	~4000 (Python + C++)
Tests	99 (30 Python + 69 C++)
Commits Git	~25

Élément	Valeur
Durée du projet	~4 sessions

Ce projet démontre des compétences en : - **Vision par ordinateur** (YOLO, estimation de distance) - **Embarqué** (C++, SocketCAN, MISRA-friendly) - **Communication industrielle** (bus CAN, DBC) - **Test logiciel** (pytest, GoogleTest) - **Sécurité fonctionnelle** (CRC E2E, FSM, timeout) - **Architecture logicielle** (séparation des responsabilités)

9. Contact

Molka Ammari Ingénieure junior en génie électrique — ENIM 2026 GitHub : <https://github.com/molkaammari/adas-can-sil>